# Parallel Gram-Schmidt Orthonormalization

Team 31
Members: 王偉俊、陳郁翔、胡佳希

# Content

- Introduction
- Implementation: CPU
- Implementation: GPU
- Profiling and Optimization
- Reference

# Introduction

# Gram-Schmidt Orthonormalization Definition

- Given a set of vectors, find an orthonormal basis of the vector space (subspace) spanned by it.
- Orthomormal basis: A basis that all the vectors in the set are **mutually orthogonal** and have **norm = 1.**

# Problem Description

- Goal: Using parallel algorithms to orthonormalize any sets of n n-dimensional vectors on both CPU and GPU.
- Function-like definition:
  - **f: {any set of n vectors in $R^n$ }⟶{orthonormal basis of the subspace spanned by the input vectors}**

# Problem Description

- Input format:
  - .txt file
  - first line: an integer **n (1 <= n <= 4096)** representing number of vectors in set
  - following n lines: each line represinting a vector of n **doubles** ranging **[-10.0, 10.0]**
  - **set of the n vectors can be either linearly dependent or independent**

# Problem Description

- Output format
  - Same as input

Example input:

```
3
1.0000 0.0 1.0
0.0 2.0 0.00000
0.0 1.00000000000 3.0
```

Example output:

```
3
0.707107 0.000000 0.707107
0.000000 1.000000 0.000000
-0.707107 0.000000 0.707107
```

# Testcase Description

- Manually created testcases: Verify correctness under small scale
- Script generated testcases: Focus more on performance measurement under large scale
  - **gen_testcase.cpp**: Given **n** and a **random seed**, generate a random testcase.
  - An easy way to create large testcases to test on performance.

# Correctness Verification

- Orthonormality Check:
  - Check whether the non-zero vectors form an orthonormal basis.
  - Represent the non-zero vectors as a matrix *Q*. $Q^*Q^T$ should be close to Identity matrix *I* if the program is correct.
- Span Preservation Check:
  - Check whether the program follows the common processing pattern shared by most GS algorithm.
  - Let input *V = (v1, v2, …, vi)*, output *Q = (q1, q2, …, qi)*. *vk* shoud be a linear combination of *q1, …, qk*.
  - **NOTE: Not all orthonormalization methods follow this pattern.**
- **check_gs.py**: Examines the output according to the input, calculates the error, and determines whether the output is correct (error < tolerance)

# Implementation: CPU

# Blocked Modified Gram-Schmidt

- Referenced algorithm: Algorithm 8 in the paper "Block Gram-Schmidt algorithms and their stability properties" made by Erin Carson et al.

**Algorithm 8** $[\mathcal{Q}, \mathcal{R}] = \texttt{BMGS}(\mathcal{X})$

1: Allocate memory for $\mathcal{Q}$ and $\mathcal{R}$
2: $[\boldsymbol{Q}_1, R_{11}] = \texttt{IntraOrtho}\,(\boldsymbol{X}_1)$
3: **for** $k = 1, \ldots, p-1$ **do**
4: $\quad \boldsymbol{W} = \boldsymbol{X}_{k+1}$
5: $\quad$ **for** $j = 1, \ldots, k$ **do**
6: $\quad\quad R_{j,k+1} = \boldsymbol{Q}_j^T \boldsymbol{W}$
7: $\quad\quad \boldsymbol{W} = \boldsymbol{W} - \boldsymbol{Q}_j R_{j,k+1}$
8: $\quad$ **end for**
9: $\quad [\boldsymbol{Q}_{k+1}, R_{k+1,k+1}] = \texttt{IntraOrtho}\,(\boldsymbol{W})$
10: **end for**
11: **return** $\mathcal{Q} = [\boldsymbol{Q}_1, \ldots, \boldsymbol{Q}_p]$, $\mathcal{R} = (R_{jk})$

Erin Carson et al., "Block Gram-Schmidt algorithms and their stability properties," arXiv, 21 Aug. 2021, arXiv:2010.12058v3, Algorithm 8, p. 15

# Blocked Modified Gram-Schmidt

1. Divide input matrix $V$ into blocks of size 64.
2. For each block $V_j$, do:
    a. Inter-Block Orthogonalization:
        i. For each block $V_k$, $k < j$, *do*:
            1. Compute correlation matrix $M = V_j * V_k^T$
            2. Update $V_j$: $V_j = V_j - M * V_k$
    b. Intra-Block Orthogonalization (standard MGS):
        i. For each vector $v_i$ in block $V_j$, do:
            1. Normalization
            2. For each vector $v_{next}$, $i + 1 <= next < block\ size$, do:
                *a.* $v_{next} = v_{next} - (v_{next}^T * v_i) * v_i$

# Inter-Block Orthogonalization

- For each block $V_k$, k < j, do:
  - **Compute correlation matrix :**

$$M = V_j * V_k^T$$

  - Update $V_j$: $V_j = V_j - M * V_k$

```
// Compute M = Block_j * (Block_k)^T
#pragma omp parallel for collapse(2) schedule(static)
for (int r = 0; r < current_bs; ++r) {
    for (int c = 0; c < k_bs; ++c) {
        double val = 0.0;
        int row_curr = j + r;
        int row_prev = k + c;
        // Standard dot product between two rows
        #pragma omp simd reduction(+:val)
        for (int x = 0; x < n; ++x) {
            val += A[row_curr * n + x] * A[row_prev * n + x];
        }
        M[r * k_bs + c] = val;
    }
}
```

# Inter-Block Orthogonalization

- For each block $V_k$, k < j, do:
  - Compute correlation matrix $M = V_j * V_k^T$
  - **Update $V_j$:**

$$V_j = V_j - M * V_k$$

```
// Update Block_j: Block_j = Block_j - M * Block_k
#pragma omp parallel for schedule(static)
for (int r = 0; r < current_bs; ++r) {
    for (int c = 0; c < k_bs; ++c) {
        double scale = M[r * k_bs + c];
        int row_curr = j + r;
        int row_prev = k + c;

        #pragma omp simd
        for (int x = 0; x < n; ++x) {
            A[row_curr * n + x] -= scale * A[row_prev * n + x];
        }
    }
}
```

# Intra-Block Orthogonalization (standard MGS)

- For each vector $v_i$ in block $V_j$, do:
  - Normalization
  - For each vector $v_{next}$, *i + 1 <= next < block size*, do:
    - $v_{next} = v_{next} - (v_{next}^T * v_i) * v_i$

```
int row_i = j + i;
double mag = std::sqrt(dot_product(A, n, row_i, row_i));

if (mag > 1e-9) {
    scale_row(A, n, row_i, 1.0 / mag);
} else {
    // Handle linearly dependent vectors (zero out)
    scale_row(A, n, row_i, 0.0);
}
```

# Intra-Block Orthogonalization (standard MGS)

- For each vector $v_i$ in block $V_j$, do:
  - Normalization
  - For each vector $v_{next}$, *i + 1 <= next < block size*, do:
    - $v_{next} = v_{next} - (v_{next}^T * v_i) * v_i$

```
// Parallelize the update of the remaining vectors in this block
#pragma omp parallel for schedule(static)
for (int next = i + 1; next < current_bs; ++next) {
    int row_next = j + next;
    double proj = dot_product(A, n, row_next, row_i);
    saxpy_row(A, n, row_next, row_i, proj);
}
```

# Other optimization

- dot_product() : vectorized
  - gets the inner product of two vectors in the matrix
- scale_row() : vectorized
  - scales a vector in the matrix
- saxpy_row(): vectorized
  - computes a = a - scale*b, where a, b are vectors in the matrix

# Discussion

- Error Accumulation
  - This algorithm involves an extremely large number of floating point operations (roughly $O(n^3)$) when problem size is huge, and each such operation will produce some error. This will eventually lead to an incorrect output.
  - Mitigation: One reorthogonalization pass, Kahan/pairwise dot products, long double, Blocked Householder QR … But all of them will impair the performance.

# Implementation: GPU

# BMGS (Blocked Modified Gram-Schmidt) On GPU

- Host: Running loops to launch jobs and solve dependency
- Device: Block-wise computation

```
// Block Modified Gram-Schmidt on GPU
for (int j = 0; j < n; j += BLOCK_SIZE) {
    int current_bs = std::min(BLOCK_SIZE, n - j);

    // 1. Inter-Block Orthogonalization:
    // Orthogonalize current block 'j' against all previous blocks 'k'
    for (int k = 0; k < j; k += BLOCK_SIZE) {
        int k_bs = std::min(BLOCK_SIZE, n - k);

        // Compute correlation matrix M = Block_j * (Block_k)^T
        dim3 grid_corr(k_bs, current_bs);
        compute_correlation_matrix_kernel<<<grid_corr, THREADS_PER_BLOCK>>>(
            d_A, gpuRes.d_M, n, j, k, current_bs, k_bs
        );

        // Update Block_j: Block_j = Block_j - M * Block_k
        dim3 grid_update(num_elem_blocks, current_bs);
        update_block_kernel<<<grid_update, THREADS_PER_BLOCK>>>(
            d_A, gpuRes.d_M, n, j, k, current_bs, k_bs
        );
    }
```

# Inter-Block Orthogonalization: Compute Correlation Matrix

- One correlation matrix element per GPU block
- Utilize all threads to compute dot product
- Reduce results to shared memory

```
__shared__ double shared_sum[THREADS_PER_BLOCK];
double sum = 0.0;

for (int x = threadIdx.x; x < n; x += blockDim.x) {
    sum += A[row_curr * n + x] * A[row_prev * n + x];
}

shared_sum[threadIdx.x] = sum;
__syncthreads();
```

# Inter-Block Orthogonalization: Update Block

```
// Update Block_j: Block_j = Block_j - M * Block_k
dim3 grid_update(num_elem_blocks, current_bs);
update_block_kernel<<<grid_update, THREADS_PER_BLOCK>>>(
    d_A, gpuRes.d_M, n, j, k, current_bs, k_bs
);
```

- A = A - M * B
- Each thread updates one element in block

```
__global__ void update_block_kernel(
    double* A, const double* M,
    int n, int j, int k, int current_bs, int k_bs
) {
    int r = blockIdx.y;  // row index in current block
    if (r >= current_bs) return;

    int row_curr = j + r;
    int x = blockIdx.x * blockDim.x + threadIdx.x;  // column index in matrix A
    if (x >= n) return;

    double update = 0.0;
    // sum over all columns in M (i.e., all rows in Block_k)
    for (int c = 0; c < k_bs; ++c) {
        double scale = M[r * k_bs + c];
        int row_prev = k + c;
        update += scale * A[row_prev * n + x];
    }

    A[row_curr * n + x] -= update;
}
```

# Intra-Block Orthogonalization

- Not much room for parallelization in the IntraOrtho part
  - Depends on previous result
- Same approach as CPU version

# Profiling and Experiment

# System spec

CPU version: Apollo Origo

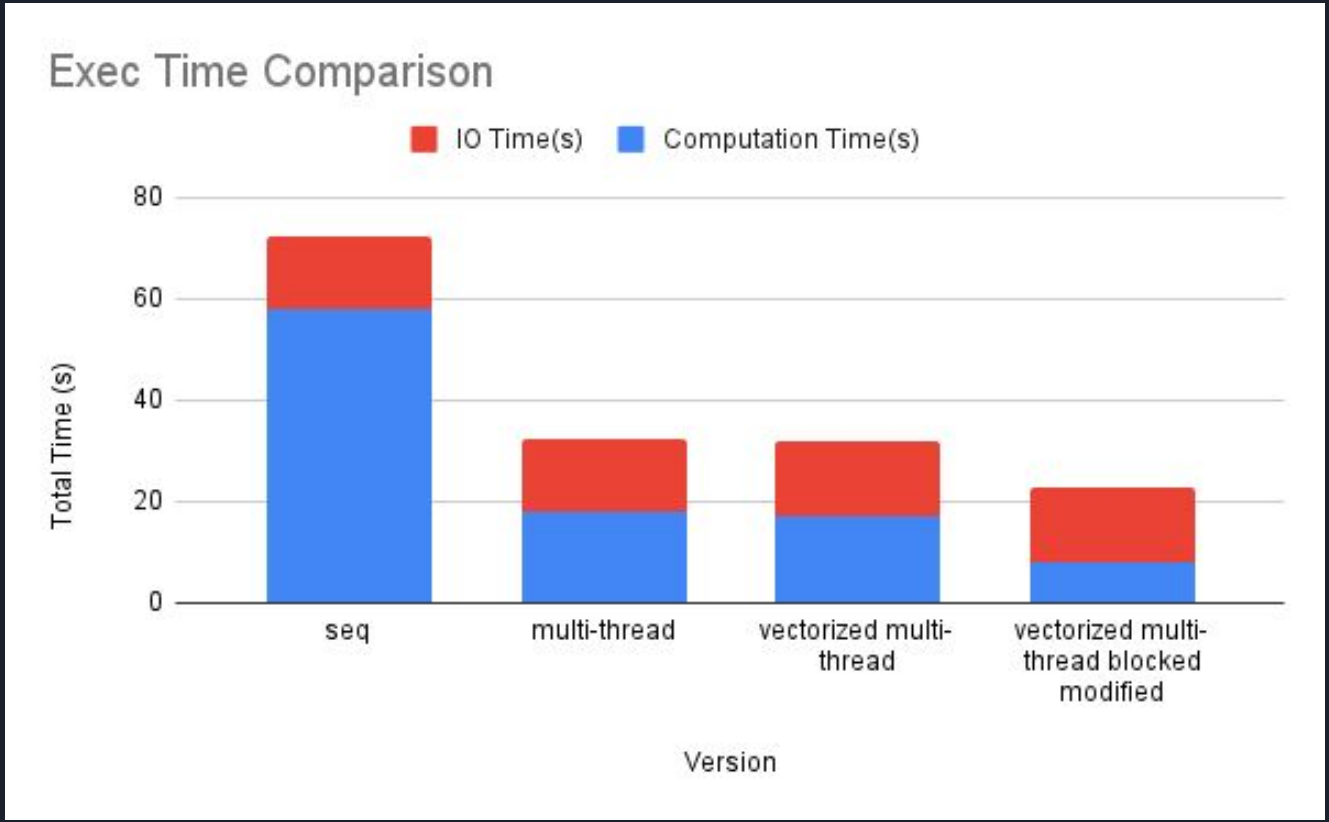
GPU version:Apollo GPU

Input data with n=4096

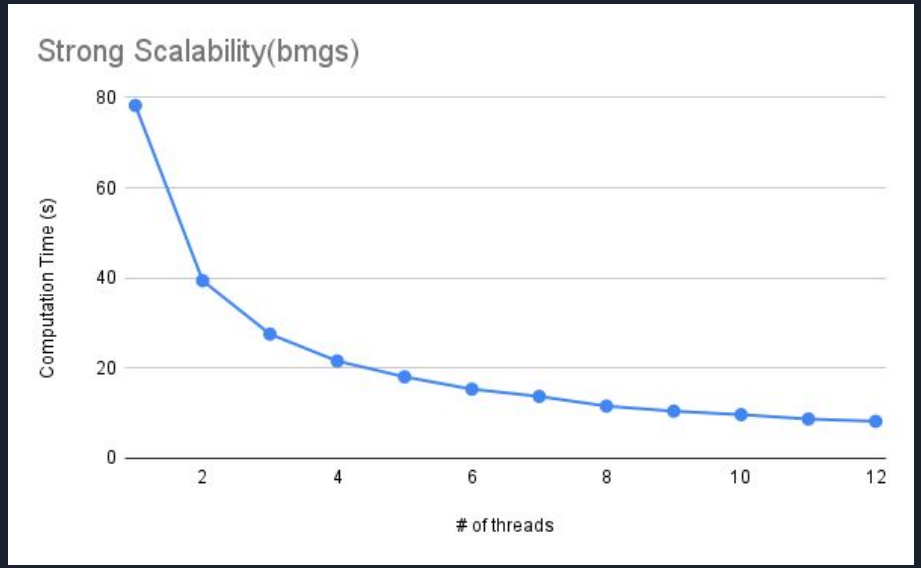
Experiment record: https://docs.google.com/spreadsheets/d/1HkA819K1HfuEwh_DcnW2t_F5M9-RHVd6El37VpDotQw/edit?usp=sharing

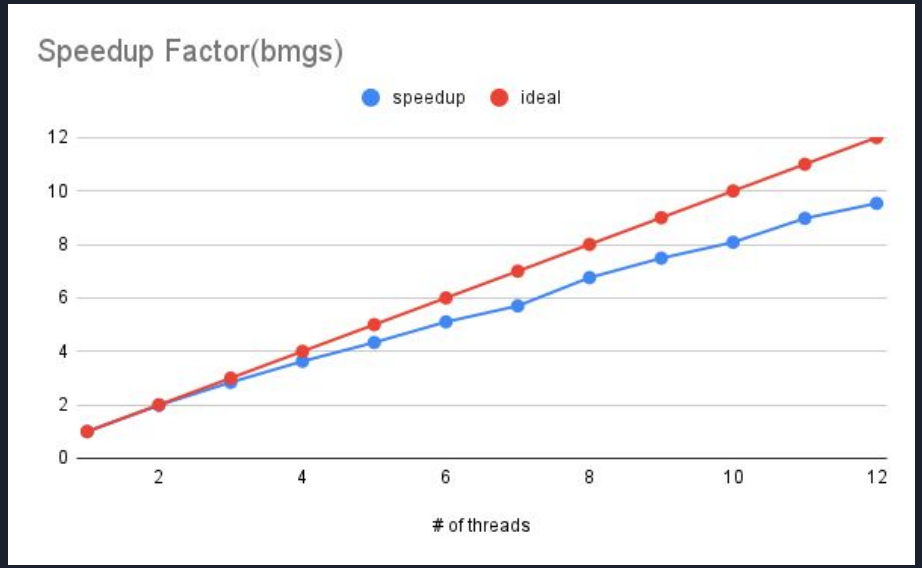
nsys_report: https://drive.google.com/drive/folders/1D9uUJ9fYstcJSdIDDs2TI_0lVGnyTC9J?usp=sharing

# CPU Version – Time Distribution



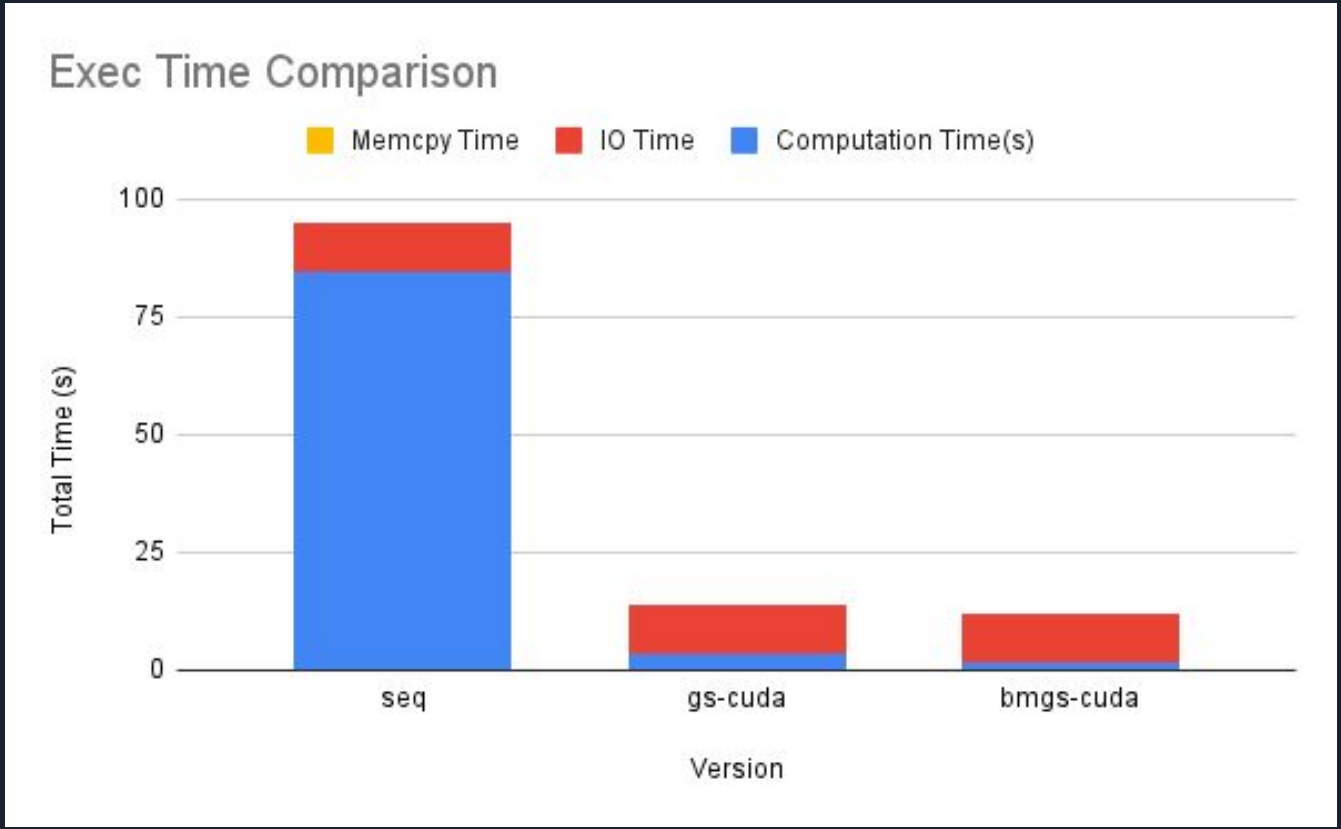
# Blocked Modified Gram-Schmidt – Strong Scalability & Speedup Factor



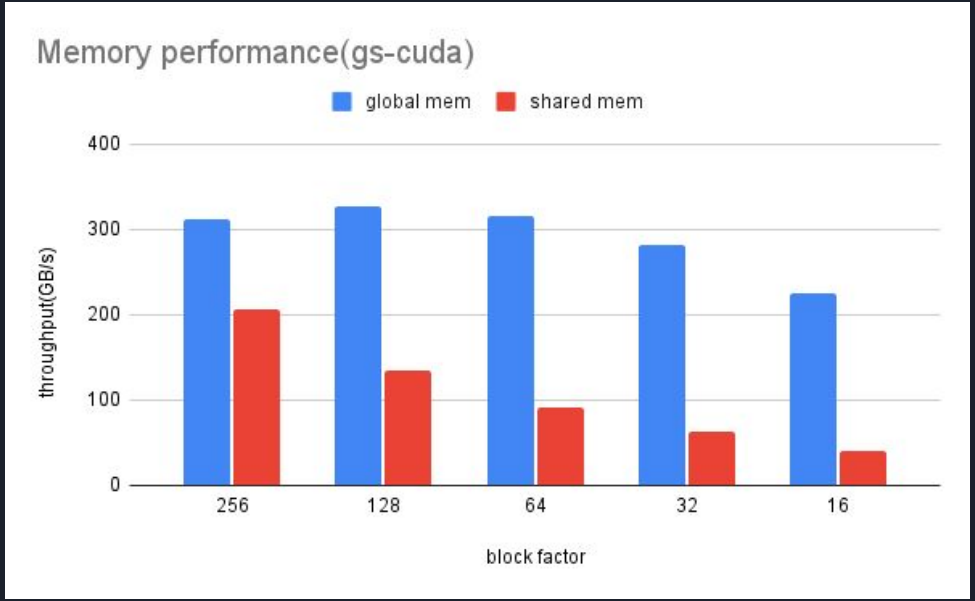



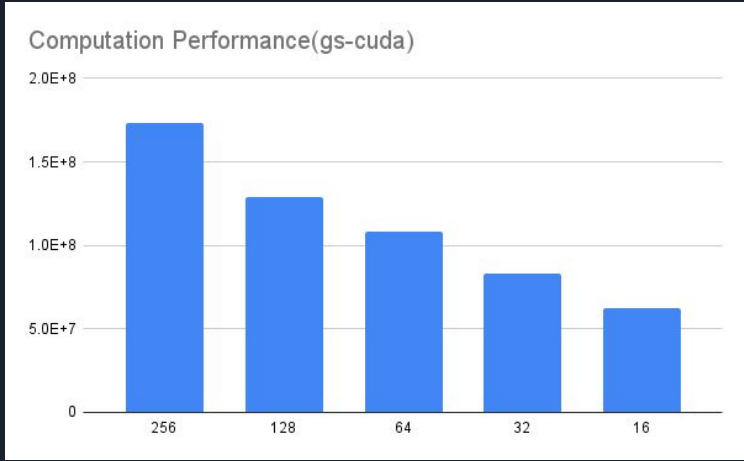
# Blocked Modified Gram-Schmidt – Load Balance

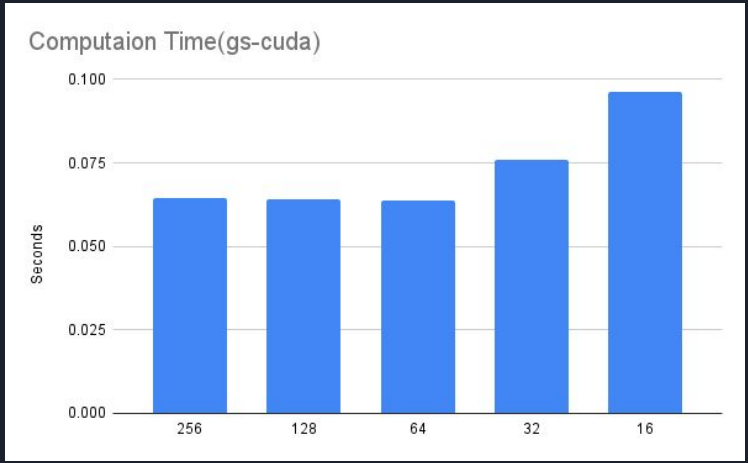
# GPU Version – Time Distribution



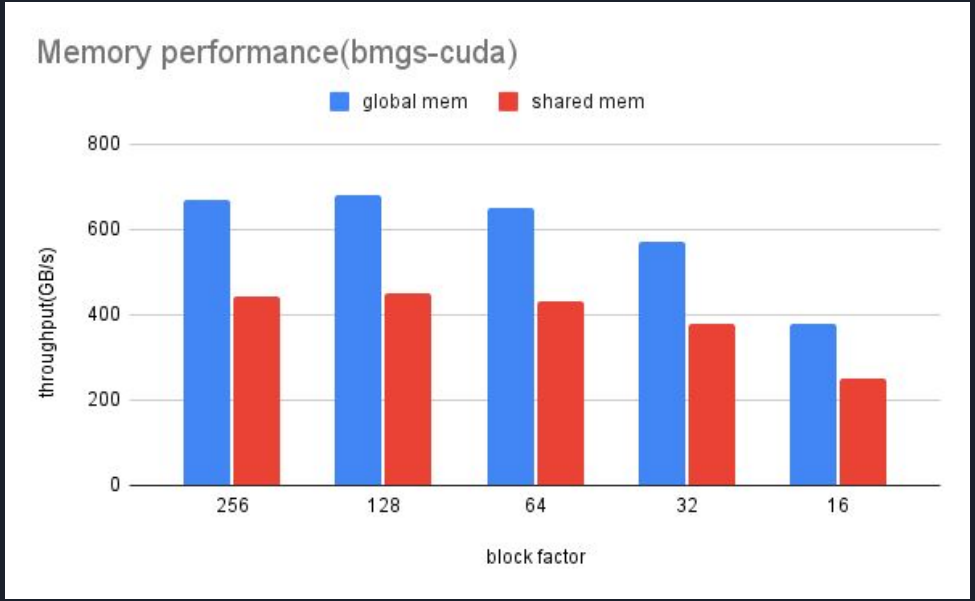
# gs cuda – Blocking Factor(n=1024)

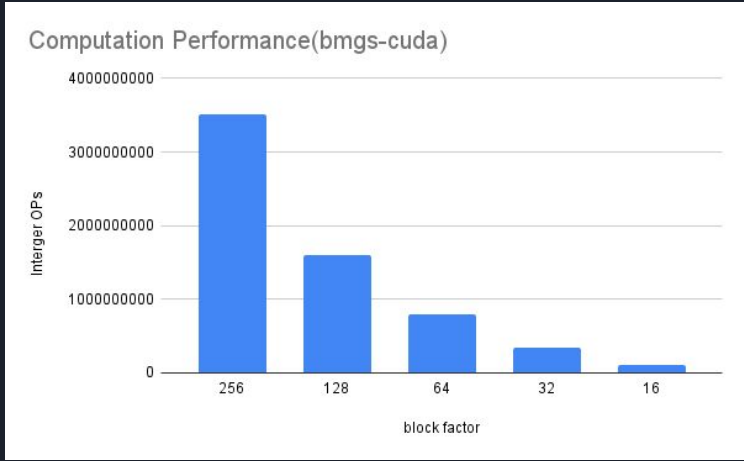


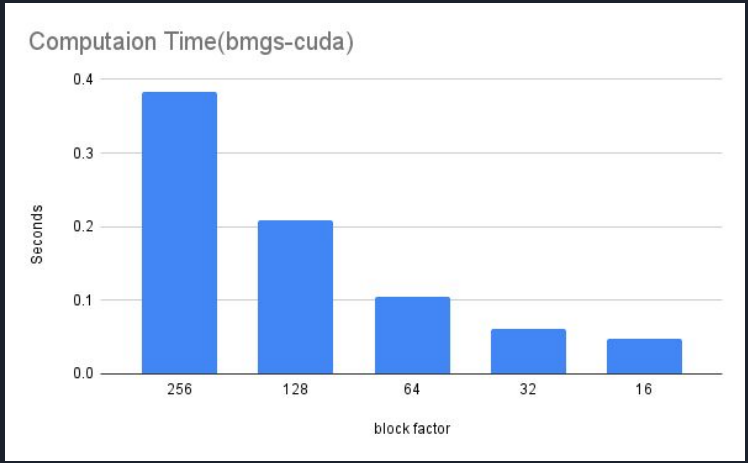



# bmgs cuda – Blocking Factor(n=1024)



# Reference


- Carson, Erin, et al. "Block Gram-Schmidt algorithms and their stability properties." arXiv, 21 Aug. 2021, arXiv:2010.12058v3.

# Thanks for listening!